Colin Fergusson
3664465
CS4745
Assignment #2

A)

Adaptive Integration:

My adaptive integration method calculates the simpson value using a, b, and the point between them, m. Then it calculates the point between a and m, and m and b and uses those to get 2 more simpson values. It adds those two together then subtracts the original simpson value before comparing the result to the tolerance, and if it is within tolerance it returns the value with possible error taken into account. If the value was not within tolerance the algorithm recurses with a being the lower bound and m being the upper bound for one of the recursions and m being the lower bound and b the upper bound of the other recursion. The tolerance gets cut in half for both recursions as they are each only half of the integral and their results are added together after finishing computation.

Task Granularity Choice:

I chose 99 for my K value because it isn't cleanly dividable by any of the process counts (1, 2, 4, 8) aside from 1, and because the value is small enough that the integrals are still too big to be within tolerance so they'll sometimes need to recure depending on the function.

Master/Worker Communication Design:

My master worker communication works by having the master probe for a send from any of the workers, then based on what the tag is it either gives the worker a new task, recieves the output from a completed task, or recieves a new task from a worker. The master is always looping through probing while keeping track of how many tasks are left and when there are none left it starts sending termination tasks to the workers until none are left at which point it returns the answer.

The workers have a list of tasks stored that they add to whenever the master gives them one, If they have none they will ask the master for a task. If there are no tasks to give out, but other processes are still working then the worker will wait until a new task is available and keep asking for tasks from the master until that is the case. When a worker isn't waiting for a task they compute their list of tasks, if they need to recure on one of their tasks they send one of the recursions back to the master and the other to their own list of tasks. Once they recieve the termination task from the master they stop searching for tasks.

Termination Strategy:

The master always keeps track of how many tasks are left, and if there are still workers working on tasks. If there are no tasks left and no more incoming tasks from workers then the master will send out a task with all values set to –1 which the workers know is the termination task. So once they receieve that task they stop searching for new tasks.

B)

Runtime, Speedup, Efficiency Tables:

My function 2 is broken in Mode 0 and I could not figure out how to fix it so the values for function 2 will be wrong.

Processes 1:

	Mode 0 1e-6	Mode 1 1e-6	Mode 2 1e-6	Mode 0 1e-8	Mode 1 1e-8	Mode 2 1e-8
Func 0 T	0.00000195s	0.00001703s	0.00000176s	0.00000668s	0.00002732s	0.00000170s
Func 0 S	N/A	0.11543956	1.10100182	N/A	0.25326827	3.98269468
Func 0 E	N/A	0.11543956	1.10100182	N/A	0.25326827	3.98269467
Func 1 T	0.00000876s	0.00002599s	0.00000278s	0.00002643s	0.00004984s	0.00000207s
Func 1 S	N/A	0.30272864	3.63795101	N/A	0.52395164	12.7124959
Func 1 E	N/A	0.30272864	3.63795101	N/A	0.52395164	12.7124959
Func 2 T	0.00000081s	0.00026297s	0.00000182s	0.00000221s	0.00056042s	0.00000200s
Func 2 S	N/A	0.00358296	0.46089809	N/A	0.00420831	1.20726021
Func 2 E	N/A	0.00358296	0.46089809	N/A	0.00420831	1.20726021

Processes 2:

	Mode 0 1e-6	Mode 1 1e-6	Mode 2 1e-6	Mode 0 1e-8	Mode 1 1e-8	Mode 2 1e-8
Func 0 T	0.00000194s	0.00001148s	0.00009611s	0.00000667s	0.00001702s	0.00007024s
Func 0 S	N/A	0.16864532	0.02507060	N/A	0.41297011	0.09891267
Func 0 E	N/A	0.08432266	0.01253535	N/A	0.20647350	0.04945633
Func 1 T	0.00000793s	0.00002928s	0.00007651s	0.00008396s	0.00004383s	0.00009596s
Func 1 S	N/A	0.42230483	0.10602198	N/A	0.61002015	0.27720134
Func 1 E	N/A	0.21115241	0.05301099	N/A	0.30501007	0.13860067
Func 2 T	0.00000087s	0.00010846s	0.00041885s	0.00000221s	0.00030572s	0.00163916s
Func 2 S	N/A	0.00867688	0.00262776	N/A	0.00806718	0.00181746
Func 2 E	N/A	0.00433844	0.00131388	N/A	0.00403359	0.00090873

Processes 4:

	Mode 0 1e-6	Mode 1 1e-6	Mode 2 1e-6	Mode 0 1e-8	Mode 1 1e-8	Mode 2 1e-8
Func 0 T	0.00000204s	0.00000917s	0.00061739s	0.00000727s	0.00001660s	0.00004324s
Func 0 S	N/A	0.22884084	0.00524603	N/A	0.44309718	0.17312800
Func 0 E	N/A	0.05721021	0.00131150	N/A	0.11077429	0.04328200
Func 1 T	0.00000980s	0.00001559s	0.00004253s	0.00009242s	0.00003864s	0.00006033s
Func 1 S	N/A	0.51724850	0.19542551	N/A	0.69871363	0.45092662
Func 1 E	N/A	0.12931212	0.04885637	N/A	0.17467840	0.11273165
Func 2 T	0.00000083s	0.00013325s	0.00035204s	0.00000235s	0.00014668s	0.00125430s
Func 2 S	N/A	0.01573951	0.00344713	N/A	0.01703997	0.00610457
Func 2 E	N/A	0.00393487	0.00086178	N/A	0.00425999	0.00152614

Processes 8:

	Mode 0 1e-6	Mode 1 1e-6	Mode 2 1e-6	Mode 0 1e-8	Mode 1 1e-8	Mode 2 1e-8
Func 0 T	0.00002057s	0.00089660s	0.00004708s	0.00000724s	0.00002341s	0.00013581s
Func 0 S	N/A	0.00414403	0.04868102	N/A	0.51857106	0.20699271
Func 0 E	N/A	0.00051800	0.00608512	N/A	0.06482138	0.02587408
Func 1 T	0.00002597s	0.00002250s	0.00010552s	0.00003703s	0.00008160s	0.00008158s
Func 1 S	N/A	0.53847749	0.11934708	N/A	0.53233670	0.33183291
Func 1 E	N/A	0.06730968	0.01491838	N/A	0.06654208	0.04147911
Func 2 T	0.00000089s	0.00004617s	0.00978192s	0.00000237s	0.00011968s	0.00147347s
Func 2 S	N/A	0.03069267	0.00020517	N/A	0.03124202	0.00223017
Func 2 E	N/A	0.00383658	0.00002564	N/A	0.00390525	0.00027877

C) Plots:

Something is wrong with my speedup calculations which I am unsure if it is because of my K value or what it could be so my data is wrong and won't be useful in a graph. On top of that my function 2 is broken in mode 0 so even more data would be wrong. On top of all of this I am already submitting this assignment late so I do not have time to make graphs, sorry.

D) Discussion:

1. Static decomposition performs poorly for adaptive workloads because the work is assigned to the different processes at the start before anything is computated and so some processes might finish really fast because they had simple integrals while others might have much larger integrals which take longer to compute, leaving the other processes waiting for it to finish.

2. Dynamic task creation improves load balance because you're never going to have multiple processes waiting on a single process for it to complete its task. Processes break down their hard tasks and give parts of them to other processes to help go through it quicker which causes less waiting and better load balancing due to processes having stuff to do more often.

3. Dynamic scheduling in MPI can be costly due to the high latency when processes are communicating with each other, which there are a lot of when the master process has lots of sends and recieves that it needs to do. There is also quite high overhead due to needing to keep track of so many things such creating, and removing tasks, and the master needing to handle multiple requests at once which it cannot do so time it wasted.

4. I am not entirely sure because my data is incorrect, but I would assume that function 2 benefited the most from dynamic scheduling and that is because the total number of accepted intervals was very high so processes needed to recure very often which would leave some processes with a lot of work and others with not as much if doing it statically. However doing it dynamically processes wouldn't have a problem with that since they are sharing the resursive work with other processes.